Word LINQ Kata 🧠🔎

Today you'll practice **LINQ basics** on a small list of words.

The goal is to practice:

- `Where`
- `Select`
- `OrderBy`
- `Count`
- clean readable chaining

Given

```
var words = new List<string>
{
    "sun", "mountain", "code", "river",
    "tree", "sky", "notebook", "cloud"
};
```

Part 1 – Filtering

Create a variable called `longWords` that contains all words with length **greater than 4**.

Part 2 – Transforming

Create a variable called `shoutWords` that contains all words in **uppercase**.

Example:

- `"sun"` → `"SUN"`

Part 3 – Filter + Transform

Create a variable called `mediumShout` that contains words with length **between 4 and 7** (inclusive), then transforms them to uppercase.

Hint: filter first, then transform.

Part 4 – Sorting

Create a variable called `byLength` that sorts the words by **length (shortest to longest)**.

Hint:

```
OrderBy(w => w.Length)
```

Part 5 – Counting

Create a variable called `longWordCount` that stores how many words have length **greater than 4**.

Hint:

```
Count(w => w.Length > 4)
```

Bonus (Optional): Build Your Own Filter

We used `Where`. Now recreate it.

Step 1 – Custom Delegate

Define your own delegate:

```
public delegate bool WordRule(string word);
```

Create:

```
public static List<string> MyFilter(
    List<string> words,
    WordRule rule)
```

Loop through the list and return matching words.

Use it to solve one of the tasks from **Part 1**.

Step 2 – Using Func Instead

Now rewrite `MyFilter` using:

```
Func<string, bool>
```

Try both versions.

Notice:

- Your method stays the same.
- Only the delegate type changes.
- This is exactly what LINQ does internally.